

oslab-project2

Environments

floppy

```
» ~/Temporary/daily → ls /dev/* > before.txt
» ~/Temporary/daily → ls /dev/* > after.txt
» ~/Temporary/daily → diff before.txt after.txt
42a43,47
> /dev/disk6
> /dev/disk6s1
> /dev/disk6s2
> /dev/disk6s3
> /dev/disk6s5
211a217,221
> /dev/rdisk6
> /dev/rdisk6s1
> /dev/rdisk6s2
> /dev/rdisk6s3
> /dev/rdisk6s5
```

Linux 设备	macOS 等价设备	说明
/dev/sdb	/dev/disk6	整个磁盘设备
/dev/sdb1	/dev/disk6s1	第一个分区
/dev/sdb2	/dev/disk6s2	第二个分区
/dev/sdb3	/dev/disk6s3	第三个分区
/dev/sdb5	/dev/disk6s5	第五个分区（逻辑分区）
/dev/sdb（直接写入原始扇区）	/dev/rdisk6	原始访问设备（更快）

minicom

安装串口驱动

```
brew install --cask ftdi-vcp-driver
```

显示串口设备

```
$ ls /dev/tty.usb* /dev/cu.usb*
/dev/cu.usbserial-1234_tul0 /dev/cu.usbserial-1234_tul1 /dev/tty.usbserial-1234_tul0
/dev/tty.usbserial-1234_tul1
```

更改 Makefile 中的设备地址

```
# DISK      = /dev/sdb
DISK        = /dev/disk6
```

```
# TTYUSB1      = /dev/ttyUSB1
TTYUSB1        = /dev/tty.usbserial-1234_tul1
```

Part1 Design Review

- `switch_to` 函数的工作机制是怎样的？是如何支持两个进程的上下文切换的？

机制：在函数调用前后指令流切换成不同的进程

切换：

- 保存现场：存 callee-saved 寄存器 (寄存器名: s*) 到 pcb 中
- 恢复现场：从栈中加载目标进程的上下文
- 你在初始化PCB和内核栈时需要做些什么事情？
分配 pid, 设置 status, 给进程分配栈空间
- 你在 `switchto_context_t` 中会保存哪些内容？
保存 GPR 中的被调用者保存寄存器

- 请介绍你设计的内核调度器 (`do_scheduler`) 的工作流程

1. 触发：进程 yield, block, 或 terminate
2. 更新进程状态
3. 选择下一个进程, RR/FCFS
4. 上下文切换

- 当一个进程被阻塞时，内核会进行如何的处理？

状态设为 BLOCK, 保存现场, 然后暂时不再调度这个进程

- 当一个进程被阻塞或获得资源时，内核会将进程的 PCB 放置在哪里？
被阻塞：放到这个资源对应的等待队列中
获得资源：把等待队列中的 PCB 放到就绪队列中
- 你设计的互斥锁机制能否支持一个进程请求多把锁？
可以，维护一个当前进程持有的互斥锁的列表

假上下文的理解：

启动时 `switch_to` 入口传入的是一个 pcb, 只会用到里面的 `kernel_stack`

task1

bug: `jmpstab=0`

本来以为是覆盖了，实际上是 `initjmpstab` 太晚了，应该放在任意一条 `bios*` 指令之前

地址范围	建议用途
0x50000000-0x50200000	Kernel 的数据段/代码段等
0x50500000-0x52000000	供内核动态分配使用的内存
0x52000000-0x52500000	用户程序的数据段/代码段等
0x52500000-0x5f000000	供用户动态分配使用的内存

地址范围	建议用途
0x5f000000-0x60000000	临时内存, 调试用

- bootblock.main 0x50200000
- kernel.main: 0x50201000

bug on 0x50201a44, do_scheduler()

exception code: 7, Store/AMO access fault, epc 50201a44, ra 50201a36

没有设置 ready_queue, 此时队列为空

bug on 0x502014b6, switch_to

exception code: 7, Store/AMO access fault, epc 502014b6, ra 50201b56

switch to 写错了, 得传入 prev.

bug: user stack 重合

crt0.S 里面 覆盖了 kernel 设置的 user_sp, 取消掉这个手动设置的 stack_top

warning: user_stack 和 kernel_stack 重合

因为 switch_to 无法分辨是 kernel 调用的 do_scheduler 调用的还是用户程序通过 sys_yield 间接调用的

解决方案: 直接在 do_scheduler 里面设置 pcb->user_sp=sp, sp=pcb->kernel_sp 表示进入内核态?

那怎么设置 init (pid=0) 呢

通过 pid 判断, 如果是 0 则不读取 ksp 而是直接设置 当前 sp 为 ksp

bug: printf 把指令也改了

分析:

```
Hardware watchpoint 2: *0x520506d2
```

```
Old value = -838762366
```

```
New value = 1344286022
```

```
_vprint (fmt=0x50203f60 "%c[%d;%dH", _va=0x52050842, output=0x50202d46 <bios_putstr>) at  
./libs/printk.c:263
```

```
263         va_copy(va, _va);
```

```
(gdb) bt
```

```
#0  _vprint (fmt=0x50203f60 "%c[%d;%dH", _va=0x52050842, output=0x50202d46 <bios_putstr>) at  
at ./libs/printk.c:263
```

```
#1  0x0000000050203732 in printv (fmt=0x50203f60 "%c[%d;%dH") at ./libs/printk.c:306
```

```
#2  0x0000000050202052 in vt100_move_cursor (x=0, y=5) at ./drivers/screen.c:20
```

```
#3 0x0000000050202456 in screen_reflush () at ./drivers/screen.c:141
#4 0x0000000050203654 in _output_wrapper (buff=0x520508fa "") at ./libs/printk.c:280
#5 0x000000005020362e in _vprint (fmt=0x52050ada "", _va=0x52050a72, output=0x5020363c
<_output_wrapper>)
    at ./libs/printk.c:272
#6 0x0000000050203682 in vprintk (fmt=0x52050ada "", _va=0x52050a72) at
./libs/printk.c:285
#7 0x00000000502036d2 in printk (fmt=0x52050ada "") at ./libs/printk.c:294
#8 0x000000005020504ca in ?? ()
(gdb) p/x $sp
$1 = 0x520506ca
(gdb)
```

sp 明显不对，什么情况？

crt0.S 改错了，还是强制设置了 sp，并且是拿 t0 设置的，t0 是 bss_start.
更正：

```
// setup stack
// la t0, USER_STACK_TOP
addi sp, sp, -16
sd ra, 0(sp)
sd sp, 8(sp)
// mv sp, t0
```

Question:

kernel 用的 stack 就是 pid0 的 stack 吗？

暂且是这样

bug: 第 23 轮时出现 \$ra=0 的情况

是因为 kernel 栈不平衡，每次减少 0x70，导致栈耗尽

栈平衡：加减对称，load/store 切换也要对称

只load不store就丢失信息了

但是其实 加载上下文的时候已经把 sp 改成 user sp 了

bug: do_scheduler 中 ra 错误

asm mv 指令写错了，覆盖了 sp

注意：ra 错误不只是内存被修改了，还可能是 sp 错了

task2

```
exception code: 7 , Store/AMO access fault , epc 502040aa , ra 50203128### ERROR ###  
Please RESET the board ###
```

list_node 没初始化 (`node->next = &node`)

mutex_release 死循环

bug: 边delete边循环导致迭代器next失效

solution: 提前存下next.

solution': do_scheduler 存 ready_queue 的时候先判断当前是否是running, 是running才转换成ready, 否则保持不变。

Part2 Design Review

? Question

请描述syscall的执行流程, 以及开发syscall时, 如何实现参数传递?

✍ Answer

执行流程:

- 用户调用 `tiny_libc::syscall()`, 执行到 `ecall`
- CPU 切换到 S-mode, 跳转到 `stvec.base`
- `stvec.base` 存的是 `exception_handler_entry`, 在此
 - 保存寄存器上下文
 - 用 `scause` 识别异常类型
 - 如果是 `ecall`, 调用 `handle_syscall`
- `handle_syscall` 中, 根据 `a7` 寄存器转发到具体函数

参数传递机制:

根据 abi, `a0-a5` 存最多 6 个参数, `a7` 存调用号

? Question

在初始化中断处理时, 需要设置哪些寄存器, 具体初始化为何值?

✍ Answer

设置 `stvec` 为 `exception_handler_entry`

设置 `sie`

? Question

当进程因异常进入内核时，需要进行哪些上下文保存？

Answer

保存通用寄存器 x1-x31

保存 sepc, sstatus

Question

开发任务3时，init_pcb_stack函数需要完成哪些初始化操作？

Answer

保存 regs.sepc = entry_point

保存 regs.sstatus = {.SPIE = 1, .SPP = U(0)}

保存 regs.sp = user_sp

保存 switchto.ra = ret_from_exception, pcb.sp = switchto.sp = kernel_sp

Question

任务3和4中，分别何时唤醒睡眠的进程？

Answer

任务3：在 do_scheduler() 中

任务4：在时钟中断处理函数中

-
- ✓ `main.c--init_syscall`
 - ✓ `main.c--init_pcb_stack`
 - ✓ `trap.S--setup_exception`
 - ✓ `irq.c--init_exception`
 - ✓ `entry.S--SAVE_CONTEXT/RESTORE_CONTEXT`
 - ✓ `entry.S--exception_handler_entry`
 - ✓ `entry.S--ret_from_exception`
 - ✓ `tiny_libc/syscall.c--invoke_syscall`
 - ✓ `syscall/syscall.c--handle_syscall`
 - ✓ `sched.c--do_sleep,do_scheduler`
 - ✓ `timer.c--check_sleeping`
-

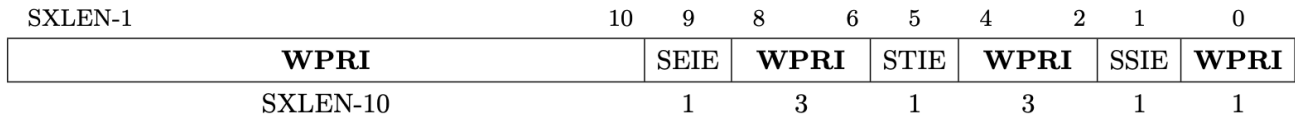


图 P2-4: Supervisor interrupt-enable register (`sie`).[6]

类型：SXLEN 位。用于控制哪些 Supervisor 模式下的中断源 被允许。

主要域：

- **SSIE** (Supervisor Software Interrupt Enable) — 允许软件触发的 S 模式中断。
- **STIE** (Supervisor Timer Interrupt Enable) — 允许定时器中断。
- **SEIE** (Supervisor External Interrupt Enable) — 允许外部设备/平台中断。
- 其余高位为保留 (WPRI = Write–Preserved, Reads Ignore) — 当前未定义，但需写入 0、读取忽略。

SSTATUS

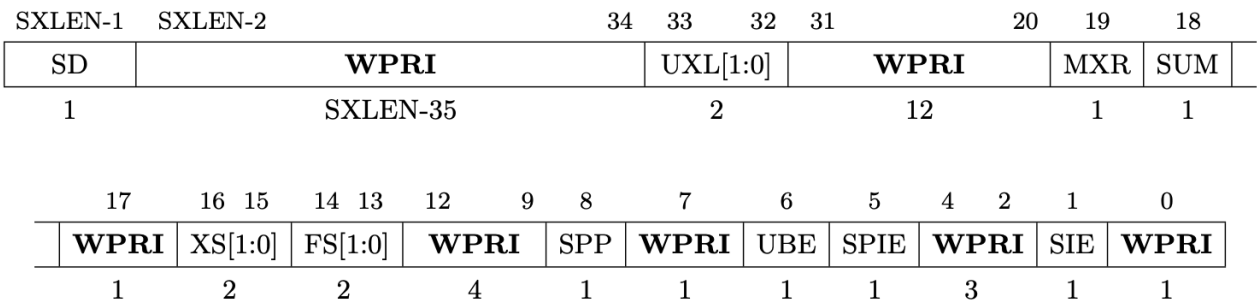


图 P2-5: Supervisor-mode status register (`sstatus`) for RV64.[6]

类型：SXLEN 位 (S 模式下的寄存器宽度，通常为 64 位)

sstatus 用于反映和控制 S 模式下的状态，如中断使能、先前特权级、用户模式下栈宽度、内存访问许可等。

- **SIE** (Supervisor Interrupt Enable) — 当处于 S 模式时，如果 SIE = 1，则允许超级模式中断。
- **SPIE** (Supervisor Previous Interrupt Enable) — 保存陷入前 S 模式中断使能状态。陷入时，SPIE ← SIE；返回 (SRET) 时，SIE ← SPIE。
- **SPP** (Supervisor Previous Privilege) — 指明陷入 S 模式前处于何特权级：0 → U 模式，1 → S 模式。返回时 (SRET) 根据 SPP 恢复相应特权级。
- **SUM** (Supervisor User Memory access) — 控制 S 模式是否允许访问用户页 (U 模式标记页)；SUM = 1 时允许。
- **MXR** (Make–eXecutable Readable) — 若 MXR = 1，则从页面属性上标记为可执行 (X=1) 的页也允许被读取。
- **UXL** (User XLEN) — 在 S 模式下可选字段，指定 U 模式寄存器/地址宽度 (如 UXLEN = 32/64)；在某些实现下只读或固定。

STVEC

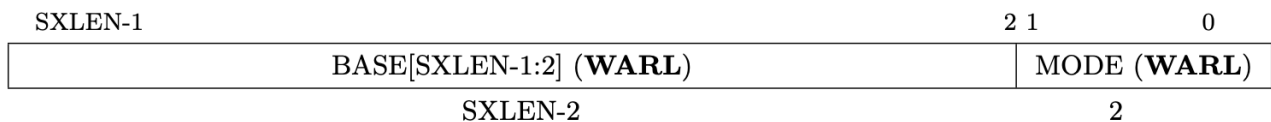


图 P2-6: Supervisor trap vector base address register (**stvec**).[6]

类型: SXLEN 位。用于指定 S 模式下陷入 (trap) 处理的入口向量。

域分解:

- **BASE** — 高位, 表示陷入向量基址 (对齐要求依照 MODE)
- **MODE** — 最低若干位 (通常两位) 表示向量模式:
 - 0 → Direct 模式: 所有陷入 (包括同步异常及中断) 跳转到 BASE。
 - 1 → Vectored 模式:
 - 对同步异常跳转到 BASE
 - 对异步中断跳转到 $\text{BASE} + 4 \times \text{cause_code}$ 。
 - ≥ 2 → 保留 (Reserved)

使用要点:

- 当陷入发生, 硬件根据 **stvec.MODE** 来设置新的 PC。
- 在 Direct 模式下, 统一入口, 内核自己分发 **cause**; 在 Vectored 模式下可加速中断处理。
- 写 **stvec** 时 BASE 必须满足对齐要求 (例如 $\text{ALIGN}=4$) 依实现规定。

SCAUSE

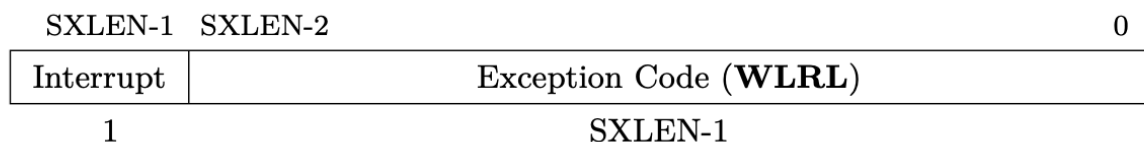


图 P2-7: Supervisor Cause register **scause**. [6]

类型: SXLEN 位。记录 S 模式下最近一次陷入 (trap) 的原因。

域分解:

- **MSB (最显位)** — 标志位: 若为 1 表示这是一个 **中断**; 若为 0 表示 **同步异常**。
- **低位其它比特** — 表示原因码 (exception/interrupt code); 例如非法指令、访问违例、定时器中断、外部中断等。
 - 若 MSB=0, **cause** 编码为异常类型;
 - 若 MSB=1, 中断类型编码通常为 $\text{cause_code} = \text{interrupt_id} + \text{constant offset}$ 。

使用要点:

- 在 trap 处理程序中, 软件可读 **scause** 来判断是中断还是异常, 并进一步分发处理。
- 硬件在陷入时自动写入 **scause**。
- 用户/内核可视为只读。

CSR	功能	关键域	注解
sstatus	描述与控制 S 模式状态	SIE, SPIE, SPP, SUM, MXR, UXL	控制中断、特权返回、用户/监督访问等
sie	中断使能控制	SSIE, STIE, SEIE	控制各类 S 模式中断源
stvec	坠入处理入口	BASE, MODE	指定陷入后 PC 跳转的规则
scause	记录陷入原因	Interrupt bit (MSB), cause code	用于判断陷入类型与原因

sscratch 用于保存 kernel tp (pcb),
操作 csr 恢复/保存上下文的时候, 保证对称性, 使用 csrrw

```
// sp <-- tp.kernel
// tp.kernel <--> sscratch(tp.user)
ld sp, PCB_USER_SP(tp)
csrrw tp, sscratch, tp
```

task4

似乎不能直接执行一个

```
while (1) {
    asm("wfi");
}
```

因为 exception_handler 依赖于 ret_from_exception 的行为 (保存 kernel tp 到sscratch)

不经过 ret_from_exception, 直接调用 exception_handler损失了对称性

可以手动执行一次 do_scheduler()

task5

调度: 根据时间片滑动窗口 (大小为), 计算每一个task的 `consumed_slice_cnt`, 然后加上一个 $1lad + 1$ 的权重, 选择目前分配时间片最少的进程分配一个时间片。

这样做有一个好处: 任务workload相差悬殊的时候, 可以保证小workload的任务在 个时间片中至少分配到一个时间片